

MemoPilot IQ: Auditable Lifecycle Governance for Persistent Agent Memory

Mahar Muavia^{*1}, Muhammad Imran Shah¹, and Tayyaba Shehzad¹

¹InventaCore AI <https://inventacore.org>

Technical report, July 2026

Abstract

Persistent assistants need more than a store of old messages. They need rules for deciding which information is current, which information has expired, and which records may enter a limited model context. We describe MemoPilot IQ, an open-source assistant whose memory layer represents conversation-derived facts as typed records with explicit lifecycle states. Retrieval combines dense and lexical signals with a transparent ranking function. A context builder then selects records under a hard 2,500-token memory budget and a default top-eight relevance window, and a per-answer trace records every inclusion and exclusion decision. Inactive records are excluded from normal retrieval while lifecycle decisions remain auditable. In a deterministic 24-scenario diagnostic, the current implementation places the required memory in context for 21 of 22 eligible scenarios, injects no stale record across 24 scenarios, and uses 31% fewer estimated historical-context tokens than replaying the setup messages. Removing lifecycle exclusion increases the stale-injection rate from 0.00 to 0.33; changing the ranking weights or priority ordering does not change the measured outcomes on this suite. The latter result limits, rather than strengthens, the claim: the diagnostic validates lifecycle handling but is too small to establish that the hand-set ranking weights are optimal. We release the implementation, scenarios, trace format, and raw evaluation artifacts.

1 Introduction

A language-model assistant can appear stateful while a conversation is open, but continuity usually depends on replaying messages inside a finite context window. Across sessions, a developer must restate project choices, constraints, deadlines, and preferences. Simply retaining every message creates a different failure: old decisions remain available after they have been replaced.

Consider a project that begins with React and Vite and later moves to Next.js. Both statements are relevant to a query about the frontend. A similarity search can retrieve either one, even though only the later decision is valid. The problem is therefore not just storage or nearest-neighbor search. The system must represent the relationship between the two statements and prevent the old one from entering the answer context.

This report treats persistent memory as a governance problem with four parts:

1. represent retained information as typed, inspectable records rather than an undifferentiated transcript;

^{*}Corresponding author: contact@inventacore.org

2. model expiry, archival, supersession, and deletion as explicit state transitions;
3. assemble a small context under a hard budget while preserving a record of what was considered; and
4. keep users able to inspect, edit, export, and erase memory.

MemoPilot IQ implements these choices in a provider-independent memory layer. In online mode, Qwen supplies structured extraction, embeddings, and answer generation. In deterministic offline mode, test doubles exercise the same storage, lifecycle, retrieval, budgeting, and trace paths without credentials. The evaluation in this report is intentionally limited to the latter mechanism path. It does not present the offline answer generator as model evidence.

The contributions are practical rather than claims of state of the art:

- a typed memory record and auditable lifecycle with directed supersession links;
- hybrid retrieval, named ranking components, and strict context budgeting with a per-answer decision trace;
- explicit boundaries for untrusted stored text, secret redaction, and user memory controls; and
- a versioned diagnostic and ablation whose raw counts are generated from the same code shipped with the report.

2 Related Work and Naming

Long-term memory for agents spans unstructured streams, hierarchical memory, structured records, and temporal or graph-based stores; recent surveys provide broader taxonomies [5, 13]. Generative Agents rank a memory stream using relevance, recency, and importance and periodically derive reflections [8]. MemGPT treats context management as paging between limited in-context memory and external storage [7]. MemoryBank applies time-dependent memory strength [14], while A-Mem organizes linked notes that evolve as new information arrives [12].

Production-oriented systems also perform memory extraction and update. Mem0 maintains selected facts in vector and optional graph stores [1]. Zep represents changing information in a temporal knowledge graph [9]. Empirical work shows why update and deletion policy matters: replaying a similar but poor prior experience can propagate errors into future behavior [11]. Preference-aware updates address a related problem in long-running conversations [10].

Two published systems use operating-system language for memory management. MemoryOS uses hierarchical short-, mid-, and long-term stores [2]; MemOS proposes a broader resource abstraction across plaintext, activation, and parameter memory [4]. The repository for MemoPilot IQ historically called its internal package “MemoryOS.” To avoid confusing that implementation label with prior work, this report uses the term *MemoPilot memory layer* and makes no claim to the MemoryOS name.

LoCoMo provides a shared benchmark for long-conversation memory and includes questions that require temporal, multi-hop, and open-domain reasoning [6]. Later work shows that factual string matching does not cover implicit constraints or cognitive memory [3]. For that reason, the internal diagnostic reported here is treated as a narrow engineering test. A complete, identically graded external comparison remains future work.

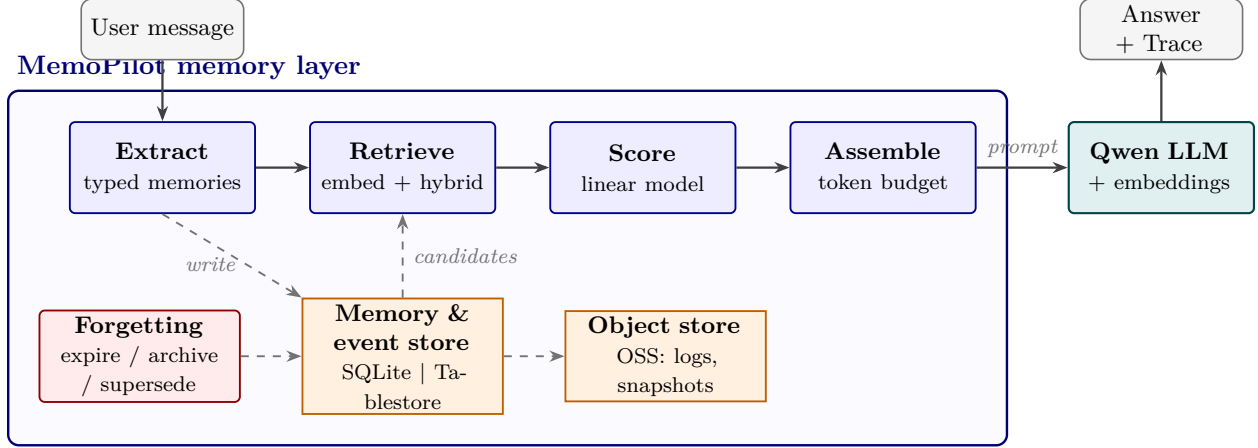


Figure 1: Architecture and request lifecycle of the MemoPilot memory layer. A user message is converted into *typed* memories (Extract), embedded and retrieved with a hybrid dense/sparse signal under structured filters (Retrieve), ranked by the interpretable linear score of Eq. 3 (Score), and packed into the prompt under a hard token budget that considers critical/pinned memories first (Assemble). The configured answer model produces the answer and a complete *Memory Trace*. A periodic forgetting sweep expires, archives, and supersedes records through auditable state transitions; memories and logs persist in the memory/event store and object store. Local mode uses SQLite and deterministic provider fallbacks; cloud adapters target Qwen, Alibaba Tablestore, and OSS.

3 System Boundary and Threat Model

The memory layer receives a user identifier, an optional project identifier, a session identifier, and a message. It returns stored-memory actions and, when answering, a bounded context plus a trace. Storage queries are scoped by user and project. API-key middleware and rate limiting protect deployed endpoints, but the memory layer itself does not establish user identity; a deployment must bind those identifiers to authenticated principals.

Stored text is treated as untrusted data. The system prompt instructs the answer model not to follow tool requests, safety overrides, or secret-exfiltration instructions found in memory. Pattern-based detectors redact common API keys, tokens, passwords, private-key blocks, and credential assignments before persistence. This reduces accidental retention but is not a data-loss prevention system and cannot detect every secret format.

Privacy labels are ranking metadata, not access control. A sensitive record receives a score penalty, but authorization must still be enforced by the application and storage layer. The evaluation does not establish resistance to prompt injection, tenant-isolation attacks, or extraction of memorized text.

4 Architecture

MemoPilot IQ is a client/server application. A React/Vite frontend calls a FastAPI backend. The memory layer owns extraction, persistence, retrieval, ranking, lifecycle transitions, context assembly, and trace generation. Figure 1 shows the request path.

The interfaces have two implementations. Local mode stores metadata and embedding vectors in SQLite and uses deterministic extraction, embedding, and answer fallbacks. Cloud mode connects

Table 1: Selected fields of a `MemoryRecord`.

Field	Role
<code>type</code>	One of 12 categories: preference, project, decision, mistake, constraint, deadline, learning goal, task, critical, temporary, outdated, or deleted by user.
<code>status</code>	Active, pinned, archived, expired, superseded, or deleted.
<code>importance, confidence</code>	Bounded extraction metadata in $[0, 1]$.
<code>privacy_level</code>	Public, private, or sensitive ranking metadata.
<code>expires_at</code>	Optional expiry for temporary or dated records.
<code>supersedes, superseded_by</code>	Directed links between an old record and its replacement.
<code>usage_count, timestamps</code>	Signals used for ranking and audit.

the same interfaces to Qwen-compatible chat and embedding endpoints, Alibaba Cloud Tablestore, and object storage. The active provider and store are exposed by the health endpoint. Cloud adapters are implemented, but this report does not claim a completed public deployment or report cloud-model results.

5 Memory Representation and Lifecycle

The atomic unit is a `MemoryRecord`. Table 1 lists the fields that affect governance.

Extraction. When a model provider is configured, a JSON-only “Memory Editor” proposes new records, updates, and forget actions. Model output is not trusted: types and privacy values are parsed against enums, importance and confidence are clamped, content and tags have length limits, and secret redaction runs again before a record is written. Offline mode uses a deterministic extractor for tests.

Supersession. The extractor compares a new record with active records in the same user and project scope. Topic-specific replacement rules and structured model actions can mark an older record **superseded** and link it to the replacement. The old record remains visible in the timeline but normal retrieval excludes it. This preserves the sequence of decisions without allowing the older choice to compete in answer context.

Expiry and archival. A sweep runs before retrieval. A record whose `expires_at` has passed becomes **expired**. An active, non-critical record is archived when it is unused, older than 30 days, and has importance below 0.35. Deletion is a separate user action. Figure 2 summarizes these transitions.

The delete endpoint defaults to a soft deletion that retains the record for audit. A hard deletion removes the memory row, but the current event log retains an audit entry. The implementation therefore does not claim cryptographic erasure or deletion from storage-layer backups.

6 Retrieval and Ranking

Retrieval first excludes archived, expired, superseded, and deleted records. For each remaining record m and query q , the implementation computes dense cosine similarity $d(m, q) \in [0, 1]$ and lexical overlap

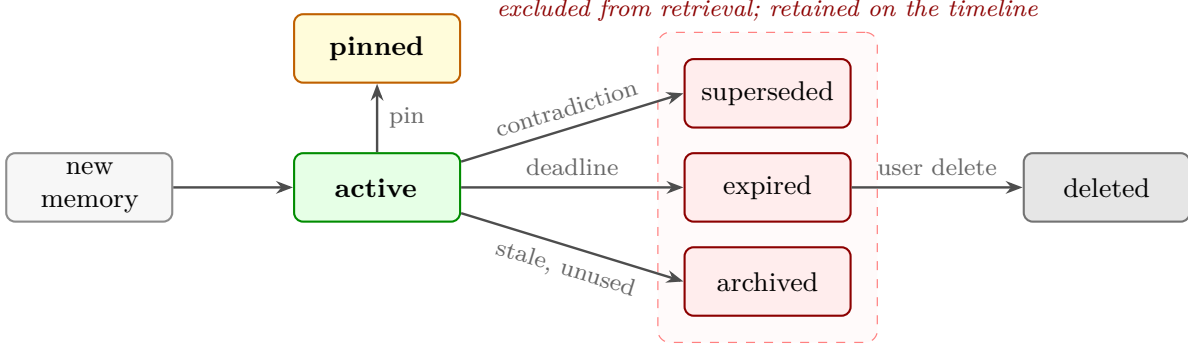


Figure 2: The **MemoryRecord** lifecycle as an auditable state machine. A new memory is born **active** (or **pinned** on user request). It is **superseded** by a contradicting decision, **expired** when its deadline passes, or **archived** when stale and unused; only an explicit user action **deletes** it. Crucially, the **superseded**, **expired**, and **archived** states are excluded from retrieval yet retained on the timeline. This keeps inactive records out of normal answer context while preserving their history for inspection.

$$k(m, q) = \frac{|T(q) \cap T(m)|}{|T(q)|}, \quad (1)$$

where $T(\cdot)$ is the set of lowercase alphanumeric tokens and $T(m)$ also includes tags. The hybrid relevance signal is

$$r(m, q) = \max \left(d(m, q), \frac{d(m, q) + k(m, q)}{2} \right). \quad (2)$$

The ranking score combines relevance with record metadata:

$$S(m, q) = 0.40r + 0.20I + 0.15R + 0.10C + 0.10U \\ + 0.15P + 0.20K - 0.30O - 0.25V - 0.50X. \quad (3)$$

I is importance, C confidence, P project match, and K a critical or pinned indicator. O , V , and X penalize outdated, private/sensitive, and superseded/expired/deleted records. Recency uses $R = \exp(-\Delta t/14)$ with Δt in days; 14 days is the exponential time constant, not the half-life. Usage is $U = 1 - (1 + \ln(1 + u))^{-1}$ for usage count u .

The score is a ranking policy, not a calibrated probability. Its weights are hand set and its positive terms can sum above one. The trace exposes each component so that a ranking can be inspected. The evaluation in Section 8 does not establish that these weights are optimal.

For stores above 1,024 eligible records, the current implementation computes dense similarity over the store and applies full hybrid scoring to the nearest 512 records plus every critical or pinned record. This is an engineering bound on reranking work, not a measured scalability result.

7 Strict Context Assembly and Trace

The context builder enforces a hard 2,500-token memory budget and a default top- k window of eight non-priority records. Fitting critical and pinned records are considered first and can raise the total above eight; other records then enter by score. Priority never overrides the budget: an oversized critical record is skipped and traced.

Table 2: Deterministic 24-scenario mechanism diagnostic.

Metric	Count	Rate/value
Target memory reaches assembled context	21/22	0.95
Scenario with stale memory injected	0/24	0.00
Selected historical-memory tokens	222	–
Replayed setup-history tokens	323	–
Historical-context reduction	–	31%

Token cost is approximated as four characters per token. This keeps behavior provider independent but is not exact for every tokenizer or language. The builder also ensures that an optional session summary fits before including it.

Every answer returns a **MemoryTrace** with the included and skipped records, ranking components, human-readable reason, estimated token cost, candidate count, and retrieval latency. The trace is available through the API and rendered in the interface. It explains the implementation’s selection path; it is not an explanation of the language model’s internal reasoning.

8 Evaluation

8.1 Questions and protocol

The repository contains 24 synthetic scenarios covering preference recall, cross-session project facts, supersession, expiry, critical constraints, mistake and deadline recall, a learning goal, and multi-fact composition. Each scenario writes one or more setup messages, runs the normal extraction and lifecycle path, asks a test question, and inspects the assembled memory context.

Twenty-two scenarios define target keywords for context recall. Fourteen define forbidden terms used to detect a stale injection. A forbidden term counts as a leak only when it appears in an included record without the expected replacement term; this avoids treating a current statement such as “Next.js instead of React” as stale merely because it names the previous choice.

The run uses deterministic extraction and hashing embeddings, SQLite storage, top- $k = 8$, and a 2,500-token memory budget. No provider key is loaded. The script `paper/run_reproducible_eval.py` generates the raw diagnostic, ablation, and manifest under `paper/results/`. Offline answer accuracy is stored for debugging but excluded from this report because the fallback is a test double rather than a language model.

8.2 Diagnostic results

Table 2 reports the current run. Context recall is 21/22. No scenario injects a stale record. The selected memory blocks contain 222 estimated tokens in aggregate, compared with 323 estimated tokens for replaying the setup messages, a 31% reduction. The comparison covers historical text only; the system prompt and current question are common to both paths.

This is a regression diagnostic, not an estimate of population performance. The scenarios are small, synthetic, and developed alongside the system. Their value is that they exercise known lifecycle cases deterministically and expose implementation regressions.

Table 3: Deterministic governance ablation. Critical inclusion is measured only when a scenario creates a critical record and that record fits the budget.

Variant	Context recall	Leak rate	Critical incl.
Full policy	0.95	0.00	1.00
No priority ordering	0.95	0.00	1.00
No lifecycle exclusion	1.00	0.33	1.00
Similarity-only weights	0.95	0.00	1.00
Uniform positive weights	0.95	0.00	1.00

8.3 Governance ablation

The ablation changes one mechanism at a time while keeping the scenarios, storage, embeddings, top- k , and budget fixed. Table 3 shows that lifecycle exclusion is decisive for stale-memory control in this suite. Allowing inactive records back into the candidate set raises the leak rate to 0.33 (8 of 24 scenarios), while context recall rises from 0.95 to 1.00 because obsolete records can now satisfy a lexical target.

The unchanged rows matter. On these scenarios, similarity already retrieves the target and critical records fit comfortably within the budget. The diagnostic therefore provides no evidence that priority ordering improves critical inclusion under pressure or that the selected weights outperform simpler alternatives. A larger ranking benchmark and adversarial budget cases are required before making either claim.

9 Limitations

No live-model result. This report does not measure answer quality with Qwen or any other hosted model. The online integration is implemented, but a versioned model-backed run from a declared deployment is still required. Provider model identifiers, prompts, temperature, retries, failures, and raw outputs must accompany that result.

Internal diagnostic. The 24 scenarios are useful regression tests but are not independent or large enough for statistical claims. They do not measure fluent generation, temporal reasoning at scale, multilingual memory, or user-perceived quality.

External evaluation pending. The repository includes a checkpointed LoCoMo adapter. This report omits a LoCoMo number because a complete run under a frozen protocol has not yet been produced. Reporting a partial run would make comparison with published systems misleading.

Hand-set policy. The ranking weights, 14-day recency time constant, 30-day archival rule, top- k , and token budget are engineering defaults. The ablation does not show that the weights are better than uniform or similarity-only ranking on the current suite.

Approximate accounting and bounded safety. The four-character token estimate can differ from the provider tokenizer. Pattern-based redaction can miss unknown credential formats, and privacy labels do not replace authorization or encryption. Prompt-injection resistance, multi-tenant isolation, and deletion guarantees require dedicated security evaluation.

Scale. The two-stage reranker bounds hybrid scoring above 1,024 records, but this report contains no throughput, tail-latency, concurrent-load, or large-store quality study.

10 Reproducibility and Availability

Source code, scenario definitions, tests, and paper artifacts are available at <https://github.com/MaharMuavia/memopilot-iq>. The project is released under the MIT License. The paper evaluation can be regenerated on Windows from the repository root with:

```
cd paper
..\backend\.venv\Scripts\python.exe run_reproducible_eval.py
```

The script blanks model-provider credentials before importing application configuration, creates an isolated SQLite database, runs the diagnostic and ablation, closes the client, removes the temporary database, and writes JSON artifacts to `paper/results`. The manifest separates the metrics cited in this report from offline answer fields that must not be interpreted as model results.

The synthetic scenarios contain no personal conversation data. The external LoCoMo dataset is not redistributed by the evaluation script.

11 Conclusion

Persistent memory fails when retrieval ignores validity. MemoPilot IQ addresses that failure with typed records, explicit lifecycle states, strict context budgeting, and a trace that makes selection decisions inspectable. The current diagnostic supports one narrow conclusion: lifecycle exclusion prevents stale records from re-entering context on the tested scenarios. It does not establish optimal ranking weights, production-scale performance, or superior answer quality. Those questions require a frozen cloud run, an external benchmark, and stronger security and scale evaluation.

References

- [1] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. URL <https://arxiv.org/abs/2504.19413>.
- [2] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. *arXiv preprint arXiv:2506.06326*, 2025. URL <https://arxiv.org/abs/2506.06326>.
- [3] Yifei Li, Weidong Guo, Lingling Zhang, Rongman Xu, Muye Huang, Hui Liu, Lijiao Xu, Yu Xu, and Jun Liu. LoCoMo-Plus: Beyond-factual cognitive memory evaluation framework for LLM agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 25085–25100. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1150. URL <https://aclanthology.org/2026.acl-long.1150/>.
- [4] Zhiyu Li, Shichao Song, Chenyang Xi, Hanyu Wang, et al. MemOS: A memory OS for AI system. *arXiv preprint arXiv:2507.03724*, 2025. URL <https://arxiv.org/abs/2507.03724>.

- [5] Jinghao Luo, Yuchen Tian, Chuxue Cao, Ziyang Luo, Hongzhan Lin, Kaixin Li, Chuyi Kong, Ruichao Yang, and Jing Ma. From storage to experience: A survey on the evolution of LLM agent memory mechanisms. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 41622–41652. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.2069. URL <https://aclanthology.org/2026.findings-acl.2069/>.
- [6] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.747. URL <https://aclanthology.org/2024.acl-long.747/>.
- [7] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL <https://arxiv.org/abs/2310.08560>.
- [8] Joon Sung Park, Joseph O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023. doi: 10.1145/3586183.3606763.
- [9] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025. URL <https://arxiv.org/abs/2501.13956>.
- [10] Haoran Sun, Zekun Zhang, and Shaoning Zeng. Preference-aware memory update for long-term LLM agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 783–793. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.38. URL <https://aclanthology.org/2026.findings-acl.38/>.
- [11] Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 623–645. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.27. URL <https://aclanthology.org/2026.acl-long.27/>.
- [12] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025. URL <https://arxiv.org/abs/2502.12110>.
- [13] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024. URL <https://arxiv.org/abs/2404.13501>.
- [14] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024. doi: 10.1609/aaai.v38i17.29946.